UNIDAD 3: UTILIZACIÓN DE LIBRERÍAS MULTIMEDIA INTEGRADAS

Módulo Profesional: Desarrollo de aplicaciones multimedia BIG DATA

Índice

RESUMEN INTRODUCTORIO 3

INTRODUCCIÓN 3

1. Conceptos sobre aplicaciones multimedia 5

2. Arquitectura del API utilizado 7

3. Descripción e instalación de las librerías multimedia 7

4. Fuentes de datos multimedia: Clips de audio, secuencias MIDI, clips de vídeo, entre otros 8

4.1 Clips de audio 9

4.2 MIDI (Interfaz Digital de Instrumentos Musicales) 9

4.3 Clips de vídeo 10

4.4 Otros 11

5. Procesamiento y reproducción de objetos multimedia 12

5.1. Reproducción de audio 12

5.2. Reproducción de vídeo 16

6. GRABACIÓN DE AUDIO Y DE VÍDEO 18

6.1. Grabación de audio 19

6.2. Grabación de vídeo 21

7. CONTROL Y MONITORIZACIÓN DE LA TRANSMISIÓN POR RTP 27

7.1. Streaming 28

7.2. API de Android 29

8. Resumen FINAL 31

RESUMEN INTRODUCTORIO

Hoy en día el desarrollo de aplicaciones móviles son una realidad en constante crecimiento. Las nuevas tecnologías avanzan a un ritmo muy elevado y, en este sentido, los contenidos multimedia juegan un papel fundamental en la implementación de software tanto para los dispositivos móviles y como para los dispositivos fijos. El desarrollo de Internet y, más concretamente, de la Web, han generado una evolución muy rápida desde la versión 1.0, a la 2.0 y la actual 3.0. La Web 2.0 supuso un cambio radical con la llegada de las webs dinámicas aportando al usuario un rol más activo y participativo, como lo es en el uso de las redes sociales.

En este sentido, son las redes sociales las que hoy en día gobierna la mayor parte de la comunicación entre los dispositivos móviles. El uso de Facebook, Twitter, Instagram y herramientas de mensajería instantánea como el famoso WhatsApp o Telegram, entre otros y aplicaciones como YouTube y Spotify son utilizadas por millones de usuarios en todo el mundo. Este nuevo concepto de la comunicación trae consigo el uso inevitable de aplicaciones y elementos multimedia, como imágenes, audio o vídeo.

INTRODUCCIÓN

En este tema se pretende aprender y abordar, de manera sencilla, conceptos relacionados con los contenidos multimedia y, a través del entorno Android Studio, conocer la forma de implementarlos en las aplicaciones Android. Se trata de un tema que resume muy bien las diferentes fuentes multimedia más conocidas y utilizadas hoy en día.

El aprendizaje de este nuevo desarrollo de contenidos, unido a lo ya aprendido en el tema anterior, dota al alumno de un mayor grado de conocimientos en la implementación de aplicaciones y programas más completos y con mayor nivel de calidad.

CASO INTRODUCTORIO

Sofía es una joven aficionada al running y a la música pop-rock. Como tal, hace uso de una aplicación de su dispositivo móvil para salir a correr con energía. Sin embargo, la aplicación de su teléfono no le permite crear listas de música con canciones aleatorias y cuando se ha bajado alguna aplicación

del Play Store le aparece mucha publicidad, incomodando la usabilidad. Para ello, le pide a su primo Juan, que posee conocimientos en informática en general y en Android Studio en particular, la creación de un reproductor de música sencillo pero que permita cumplir los requisitos de Sofía, es decir, ausencia de publicidad y generar una lista de canciones para que se reproduzcan de forma aleatoria y no por orden de aparición.

Para Juan, es un bonito desafío capaz de desarrollar en alguna que otra tarde y, además, piensa en subirlo al Play Store para obtener 0,99 € por descarga realizada. La esencia de este ejemplo práctico es la capacidad de implementar contenido multimedia en Android, en este caso, a través de música.

Si Juan es capaz de realizar este reproductor, ¿por qué no va a ser capaz de hacer una nueva versión, esta vez para la reproducción de vídeos? Sea como fuere, Juan estará agradecido a Sofía por este nuevo reto que se plantea cumplir.

1. CONCEPTOS SOBRE APLICACIONES MULTIMEDIA

Las aplicaciones multimedia representan un medio de gran calado en los usuarios tanto de los dispositivos móviles como en cualquier otro sistema informático. Esto es debido a que, tanto visualmente como funcionalmente, ofrecen un alto grado de interactividad y usabilidad en un gran número de aplicaciones. En muchos casos, este tipo de aplicaciones son parte esencial del ocio de los usuarios y, como tal, forman parte de su día a día.

Las aplicaciones multimedia, se caracterizan por presentar en sus interfaces componentes visuales tales como textos, sonidos, iconográficos, imágenes estáticas y dinámicas y vídeo, entre otros. Las siguientes líneas resumen estos componentes visuales que van a aparecer de alguna u otra forma en el desarrollo de interfaces de usuario.

- **Texto**

Los textos son parte esencial de las aplicaciones ya sean multimedia o no. Como se estudió en el tema anterior, los textos aparecen en forma de labels o etiquetas, y pretenden informar al usuario sobre algún aspecto concreto de la aplicación. En este sentido, una canción, un álbum y un artista que se está reproduciendo en una aplicación multimedia de un dispositivo móvil se representará con un texto. Como tal, no es un componente considerado típicamente multimedia, pero sí que en la mayoría de los casos se requiere para describir algo determinado.

- ***Sonido***

A diferencia de los textos, los sonidos sí son considerados elementos típicamente multimedia. Se trata de un componente de tipo no visual pero sí auditivo, que estará asociado a algún componente visual. En este sentido, es posible imaginar una aplicación multimedia tipo Spotify en la cual estará presente el sonido de una canción una vez se pulse sobre el resultado de una búsqueda o sobre un botón Play (reproducir). Además, los sonidos también se encuentran incorporados en la reproducción de vídeos.

- **Iconográficos**

Un elemento iconográfico es utilizado para una representación concreta a través de una imagen descriptiva. En este sentido, los típicos iconos de las

aplicaciones una vez instaladas, son ejemplos claros de este componente. Entre los ejemplos más comunes se nombran los siguientes: una nota musical para la representación de un reproductor de música, un sobre para la representación de una aplicación de correo o una cámara de fotos para representar la aplicación para realizar fotografías. Estos componentes se encuentran en multitud de aplicaciones multimedia; basta recordar los típicos botones de play, pause o stop para reconocer su función.

- **Imágenes estáticas e imágenes dinámicas**

Al igual que los iconográficos, las imágenes forman parte de un sinfín de aplicaciones tanto multimedia como de otra índole. La diferencia principal entre imágenes estáticas y dinámicas reside en que las primeras están formadas por una única imagen y las segundas por varias. En este sentido, una imagen estática puede ser la portada de un disco de música que puede verse en el fondo de un reproductor de música y una imagen dinámica puede ser un GIF de un altavoz formado por 3 imágenes reproduciéndose continuamente, simulando la escucha de una canción.

- **Vídeo**

El vídeo, al igual que el sonido, es un elemento multimedia muy utilizado en la actualidad. No hay más que explorar la web para ver en primera persona el avance exponencial que este tipo de componente ha experimentado en los últimos años. La publicidad, las redes sociales, las aplicaciones de mensajería y la influencia de webs populares como el caso de YouTube han servido para que los vídeos se hayan instalado en la sociedad como un elemento más en la comunicación diaria, formando parte de todos y cada uno de los usuarios, fundamentalmente de dispositivos web y móviles.

2. ARQUITECTURA DEL API UTILIZADO

Una API (Application Programming Interface) es un conjunto de funciones, métodos y procedimientos que ofrece una biblioteca para ser usada por otro software. En este sentido, el Framework multimedia proporciona un API cuya arquitectura está formada por varios paquetes. Los paquetes se componen de cuatro clases principales, dos de las cuales permiten reproducir contenidos multimedia como son *MediaPlayer* y *JetPlayer*. Las otras dos clases permiten grabar contenidos como audio, vídeo y fotos usando el hardware del dispositivo. Estas clases son *MediaRecorder* y *Camera*.

ENLACE DE INTERÉS

En el siguiente enlace tenemos la referencia oficial de las librerías de MediaPlayer con ejemplos muy explicativos.

https://developer.android.com/guide/topics/media/mediaplayer?hl=es

3. DESCRIPCIÓN E INSTALACIÓN DE LAS LIBRERÍAS MULTIMEDIA

La librería para manejar contenidos multimedia está organizada en los siguientes paquetes:

- **Android.media:** contiene las clases para reproducir y grabar contenidos multimedia de audio y vídeo. Dentro existe otro paquete llamado android.media.audiofx que contiene un conjunto de clases para gestionar efectos de audio.

  Algunos contenidos multimedia de audio que permite manejar son, por ejemplo, MP3 u otros formatos musicales, tonos de llamada, efectos de sonido de juegos y tonos DTFM (Dual-Tone MultiFrecuency, marcación por tonos). También permite manejar vídeo para grabar o para reproducir un streaming desde la Web o desde un almacenamiento local.

- **Android.media.efect:** proporciona varias clases que permiten aplicar una variedad de efectos visuales a imágenes y vídeos como por ejemplo eliminar el efecto de los ojos rojos.

  Esta librería maneja efectos visuales a imágenes y vídeos como por ejemplo eliminar el efecto de los ojos rojos, convertir una imagen a una escala de grises, ajustar brillo, ajustar saturación, rotar la imagen, aplicar el efecto de ojo de pez, etc. Los efectos son aplicados directamente a texturas OpenGL, con lo cual se debe tener un contexto OpenGL válido antes de poder usar los efectos de este paquete.

ENLACE DE INTERÉS

En el siguiente enlace encontraremos documentación oficial sobre el Audio y el vídeo dentro de Android Studio.

https://developer.android.com/guide/topics/media?hl=es

4. FUENTES DE DATOS MULTIMEDIA: CLIPS DE AUDIO, SECUENCIAS MIDI, CLIPS DE VÍDEO, ENTRE OTROS

En esta sección se analizan las fuentes de datos multimedia más conocidas hoy en día, como clips de audio y vídeo, y secuencias tipo MIDI, entre otros.

COMPRUEBA LO QUE SABES

¿Qué tipos de texturas OpenGL existen? Puedes ayudarte de Internet para responder a la pregunta.

4.1 Clips de audio

Un clip es un segmento pequeño de contenido multimedia que puede ser solo de audio (audioclip) o de vídeo (videoclip). Se ha de diferenciar entre: formato del fichero, que es el formato de los datos de audio con el que se guarda en el fichero, y el códec de audio, que realiza la codificación y descodificación de los datos de audio almacenados en el fichero y es utilizado por los programas reproductores. Existen varios formatos de audio:

- **Formatos de audio sin compresión.** Son ficheros en los que se guarda la información de audio sin ningún algoritmo para comprimirlos. Algunos de estos son por ejemplo WAV, AIFF, AU, etc.

- **Formatos de audio sin pérdida.** La información de audio es comprimida en el fichero y se puede recuperar el contenido sin pérdida de partes de este. Algunos de estos son FLAC, Monkey´s Audio, WavPack, etc.

- **Formatos de audio con pérdida.** La información de audio es comprimida de tal forma que se codifica utilizando la mínima cantidad de información posible siendo imposible la reconstrucción exacta de los datos originales. Algunos son: MP3, Vorbis, Musepack, AAC, etc.

4.2 MIDI (Interfaz Digital de Instrumentos Musicales)

MIDI es un protocolo de comunicación de serie estándar para comunicar y compartir información en la generación de sonidos entre computadores, sintetizadores, secuenciadores, controladores y demás equipos electrónicos de tratamiento de sonido. No contiene música, sino instrucciones que cuando son interpretadas por un sintetizador producen sonidos.

ENLACE DE INTERÉS

A través del siguiente enlace se puede conocer más en detalle acerca del protocolo de comunicación MIDI.

http://www.disca.upv.es/adomenec/IASPA/tema5/Midi.html

4.3 Clips de vídeo

Un videoclip es una colección de imágenes acompañadas de sonido. No exige de un proceso de producción y edición industrial o técnico, siendo normal que la información de audio y vídeo se suela grabar en pistas separadas y luego se combinen en su reproducción. Existen varias fuentes multimedia:

- El formato MPEG proporciona una compresión de datos con pérdida (se usa en vídeo). Se distinguen cuatro tipos:

  - **MPEG-1:** es estándar inicial de audio y vídeo para CD.

  - **MPEG-2**: ofrece una mayor calidad con un mayor ancho de banda (entre 3 y 10 Mb/s). Se utiliza para difundir audio y vídeo de calidad para televisión.

  - **MPEG-3:** propuesto originalmente para la Televisión de Alta Definición (HDTV). Se ha abandonado posteriormente a favor de MPEG-2.

  - **MPEG-4**: expande MPEG-1 para soportar objetos de audio/vídeo, contenidos 3D y gestión digital de derechos.

- Los formatos más comunes para vídeo son MOV y AVI.

4.4 Otros

Formato contenedor: Permite almacenar información de vídeo, audio, texto de subtítulos y de capítulos, metadatos e información de sincronización. Existen varios formatos contenedores: AVI, MPG, MOV (contenedor de QuickTime), ASF (contenedor de WMV y WMA), Ogg, OGM, RMVB y Matroska.

Formato para imágenes: Como pueden ser los siguientes:

- Formatos de impresión (EPS, TIFF, PSD…)

- Formatos para la Web (JPG, GIF, PNG, BMP…): JPG tiene una buena calidad de imagen y una buena velocidad de descarga y visualización. GIF proporciona una calidad media de imagen y alta velocidad de descarga y visualización. PNG proporciona una excelente calidad de imagen, pero tiene una baja velocidad de descarga. BMP tiene escasa compresión, pero está muy presente ya que casi todos los PC lo soportan.

VIDEO DE INTERÉS

En el vídeo encontrarás información sobre las transiciones entre Activities.

https://www.youtube.com/watch?v=aqSVX50bZas

5. PROCESAMIENTO Y REPRODUCCIÓN DE OBJETOS MULTIMEDIA

En esta sección se estudia la implementación para la reproducción tanto de audio como de vídeo.

5.1. Reproducción de audio

Como se describió anteriormente, el sistema operativo Android proporciona en su API multimedia la clase *MediaPlayer*, la cual permite reproducir audio, entre otros medios. Para ello, es importante acceder al fichero manifest e incluir en él fichero las declaraciones de permiso de uso.

Los usos son los siguientes:

- **Permiso para acceso a Internet:**
  <uses-permission android:name="android.permission.INTERNET"/>

- **Permiso de desbloqueo:**
  <uses-permission android:name="android.permission.WAKE_LOCK"/>

La clase MediaPlayer permite reproducir audio:

- **Desde la Web:** el contenido está ubicado en una fuente accesible por una URL de Internet.

- **Desde un fichero local:** el contenido está ubicado en un fichero que es empaquetado como parte del fichero apk de la aplicación. El fichero multimedia debe ser incluido como una fuente en la carpeta /res/raw o en /assets.

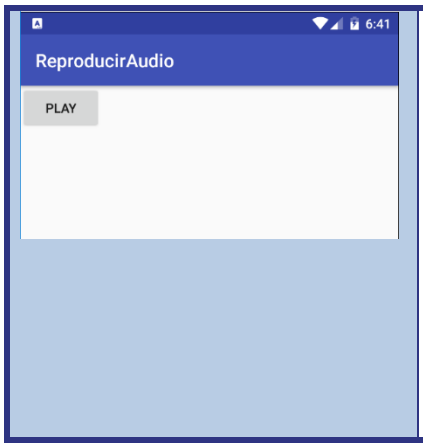
- **Desde una tarjeta SD (Secure Digital):** el contenido reside en una tarjeta SD en el dispositivo móvil

Los formatos que soporta Android son los siguientes:

AUDIO	AAC LC/LTP, HE-AACv1 (AAC+), HE-AACv2 (enhanced AAC+), AMR-NB, AMR-WB, FLAC, MP3, MIDI, Vorbis, PCM/WAVE
IMAGEN	JPEG, GIF, PNG, BMP, WEBP
VÍDEO	H.263, H.264 AVC, MPEG-4 SP, VP8

Un objeto MediaPlayer sigue un ciclo de vida definido por un conjunto de estados y transiciones entre estados. El ciclo de invocación de procedimientos habituales es, por este orden, el siguiente:

- new MediaPlayer()
- setDataSource()
- prepare ()
- start()
- pause ()
- stop()

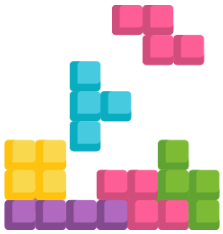



```
public class MainActivity extends AppCompatActivity {

 Button play;

 @Override
 protected void onCreate(Bundle
 savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main);

 play = (Button)
 this.findViewById(R.id.button);
 }
}
```



EJEMPLO PRÁCTICO

REPRODUCCIÓN DE AUDIO DESDE LA WEB

Se va a realizar un ejercicio práctico para la reproducción de un audio desde la Web. Para ello, se va a crear un nuevo proyecto y se va a añadir un botón llamado PLAY, tal y como se muestra en la figura adjunta. El objetivo de este ejercicio es reproducir una canción ubicada en Internet una vez se realice clic sobre el botón PLAY.

Para reproducir audio, en primer lugar, se debe declarar un objeto MediaPlayer, importándolo previamente del paquete android.media:

```
import android.media.MediaPlayer;

MediaPlayer mediaPlayer;
mediaPlayer= new MediaPlayer();
```

Para la reproducción de contenidos en la web, se debe usar el método setDataSource (). Posteriormente, es necesario preparar el objeto usando el método prepare () y, a continuación, invocar al método start (), el cual inicia la reproducción. La reproducción del audio ha de llevarse a cabo bajo el evento del botón al hacer clic en él (PLAY).

```
final MediaPlayer mediaPlayer;
mediaPlayer= new MediaPlayer();

final String CANCION=
"http://sss.moda.pe/audio/despacito-de-luis-fonsi-ft-
daddy-yankee-7c24e.mp3";

play.setOnClickListener(new View.OnClickListener() {
 @Override
 public void onClick(View v) {
 try {
 mediaPlayer.setDataSource(CANCION);
 mediaPlayer.prepare();
 mediaPlayer.start();
 } catch (IOException e) {
 e.printStackTrace();
 }
 }
});
```

A la hora de implementar los métodos para la reproducción del audio a través de la web, Android Studio pide hacerlo bajo un try-catch para capturar cualquier posible error que se pueda producir.

Se propone al alumno implementar este ejemplo y trabajar con él en los casos prácticos propuestos, como por ejemplo crear un nuevo botón y generar un evento en él para poder parar la reproducción del audio.

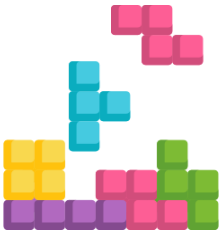
Para ello, se hace uso del método créate () de MediaPlayer. En su llamada, se le pasa como parámetro el fichero que contiene el audio y la llamada al

método créate () ya incluye la invocación a prepare (), con lo cual no es necesario hacerla. Sólo falta reproducir el audio a través de la invocación al método start ().

```
final MediaPlayer mediaPlayer;
mediaPlayer= MediaPlayer.create(this, R.raw.cancion);

play.setOnClickListener(new View.OnClickListener() {
 @Override
 public void onClick(View v) {
 mediaPlayer.start();
 }
});
```

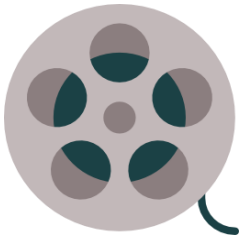
Una vez que el objeto está reproduciendo el contenido se puede parar mediante el método stop (). También se puede poner en pausa mediante el método pause (), permitiendo restablecer la reproducción invocando start ().



EJEMPLO PRÁCTICO

REPRODUCCIÓN DE AUDIO DESDE FICHERO

Se va a realizar un ejercicio práctico para la reproducción de un audio desde un fichero. Para ello, se va a crear un nuevo proyecto y se va a añadir un botón llamado PLAY al igual que en el ejercicio práctico 1. El objetivo de este ejercicio es reproducir una canción ubicada en res/raw una vez se realice clic sobre el botón PLAY. En este ejemplo se obtiene un fichero MP3 llamado cancion.mp3 y se ubica en res/raw para posteriormente implementar la reproducción.



VIDEO DE INTERÉS

En el siguiente enlace tendréis un vídeo sobre la reproducción de sonidos y audio.

https://www.youtube.com/watch?v=_rxpGKUkClg

5.2. Reproducción de vídeo

Para la implementación de la reproducción de vídeo es necesario hacer uso de las siguientes clases:

- **VídeoView**: Permite visualizar un fichero de vídeo, hereda de la clase View, permitiendo su reproducción desde la web o desde un fichero de vídeo.

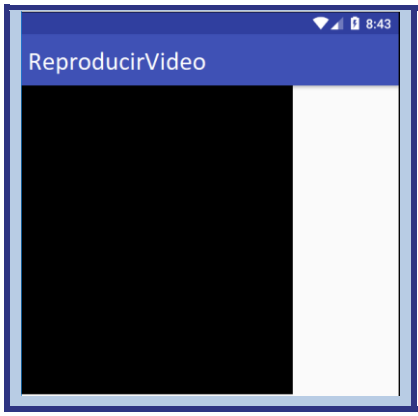
- **MediaController**: Se combina con VídeoView para el manejo de la reproducción del vídeo, enlazándose a través del método setMediaController. La clase MediaController proporciona los botones típicos de PLAY, STOP, PAUSA, AVANCE, RETROCESO, etc.

A través del método setVídeoURI se establece la fuente que contiene el vídeo a reproducir. Este método pide como parámetro un objeto de tipo Uri (dirección web donde se aloja el vídeo).

EJEMPLO PRÁCTICO

REPRODUCCIÓN DE VÍDEO DESDE FICHERO DE VÍDEO

Se va a realizar un ejercicio práctico para la reproducción de un vídeo desde un fichero de vídeo. Para ello se va a crear un nuevo proyecto y se va a añadir un componente VídeoView, tal y como se muestra en la figura adjunta. Además, se va a añadir la clase MediaController para poder "jugar" con el vídeo avanzando, retrocediendo, reproduciendo y parando. En este ejemplo se obtiene un fichero VP8 llamado vídeo.webm y se ubica en res/raw para posteriormente implementar la reproducción.




```
public class MainActivity extends
AppCompatActivity {

VídeoView vídeo;

@Override
protected void onCreate(Bundle
savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

vídeo = (VídeoView)
this.findViewById(R.id.vídeoView);
```

Se procede a la implementación de este reproductor a través de las siguientes líneas de código:

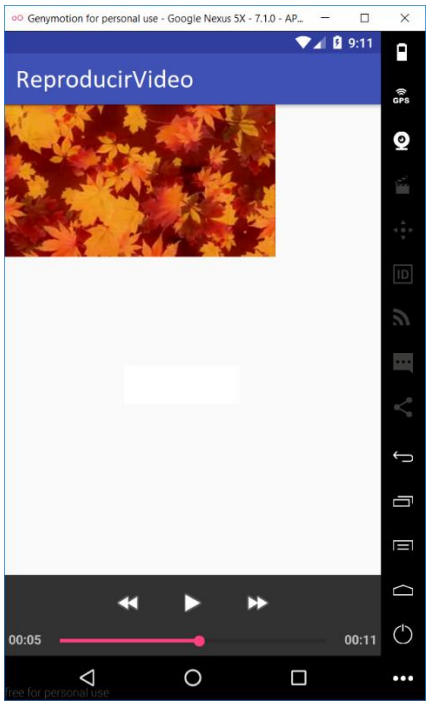
```
public class MainActivity extends AppCompatActivity {

VídeoView vídeo;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

vídeo = (VídeoView)
this.findViewById(R.id.vídeoView);

MediaController mController = new
MediaController(this);
vídeo.setMediaController(mController);
vídeo.setVídeoURI(Uri.parse("android.resource://" +
            getPackageName() +"/"+R.raw.vídeo));
vídeo.requestFocus();
 }
}
```

El resultado tras la ejecución de este código se muestra a continuación. Puede comprobarse cómo existen 2 componentes media distintos, por un lado, el componente VídeoView, necesario para la reproducción del vídeo y, por otro lado, el componente MediaController, para reproducir, pausar o mover el vídeo en cuestión.



En la imagen se muestra el vídeo en pausa, puesto que el componente MediaController presenta el PLAY activo, para reanudar la reproducción. En caso contrario, hubiera presentado el PAUSE activo.

6. GRABACIÓN DE AUDIO Y DE VÍDEO

En esta sección se estudia la implementación para la grabación tanto de audio como de vídeo. Para ello, se va a hacer uso de la clase *MediaRecorder*. Hay que tener en cuenta que existen varias diferencias entre la grabación de vídeo y de audio: la grabación de vídeo exige un procedimiento de visualización y acceso a la cámara del móvil, lo que implica algo más de complejidad que en la grabación del audio.

El ciclo de vida habitual comienza con el estado Inicial. Mediante el método setAudioSource o setVídeoSource, se establece la fuente de audio o de vídeo, respectivamente, para ser usada en la grabación (el micrófono, la cámara…).

Consideraciones para tener en cuenta:

- El método setOutputFormat define el formato del fichero de salida que se produce durante la grabación.
- Se suena usar un formato 3GP cuando se está usando un codificador de vídeo H.263 y un formato AMR si se usa un codificador de audio.
- El método setAudioEnconder configura el codificador.
- El método prepare () prepara el objeto MediaRecorder listo para comenzar a capturar y codificar los datos a grabar.
- El método start () se pasa al estado Recording.
- Los métodos stop () o reset () detienen la grabación y se vuelve al estado Initial, quedando aún el objeto MediaRecorder listo para su reutilización.
- El método reléase () libera el objeto y ya no se puede volver a usar siendo necesaria su construcción nuevamente.

6.1. Grabación de audio

Para la grabación de audio es necesario modificar el fichero de manifiesto de la aplicación de la siguiente manera:

```
<uses-permission
android:name="android.permission.RECORD_AUDIO"/>
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

```
public class MainActivity extends AppCompatActivity {

Button grabar;

@Override
protected void onCreate(Bundle
savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

grabar = (Button)
this.findViewById(R.id.button2);
 }
}
```

Se procede a la implementación a través de las siguientes líneas de código:

```
final MediaRecorder recorder = new MediaRecorder();

grabar.setOnClickListener(new View.OnClickListener() {
@Override
public void onClick(View v) {
recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
recorder.setOutputFile(PATH_NAME);

  try {
recorder.prepare();
recorder.start(); // comienza la grabación ...
 } catch (IOException e) {
e.printStackTrace();
 }

 }
});
```

En PATH_NAME, se dejará la ruta al fichero donde se almacenará la grabación del audio. Con los métodos stop (), reset () y release () se para la grabación, se reutiliza el objeto y se libera el objeto, respectivamente.

EJEMPLO PRÁCTICO

GRABACIÓN DE AUDIO

Se va a realizar un ejercicio práctico para la grabación de audio. Para ello se va a crear un nuevo proyecto y se va a añadir un componente Button llamado GRABAR AUDIO, tal y como se muestra en la figura adjunta. El objetivo es que al pulsar sobre el botón se proceda a la grabación del audio a través del micrófono

VIDEO DE INTERÉS

En el vídeo encontrarás información sobre la reproducción de sonidos en una aplicación Android

https://www.youtube.com/watch?v=12xhoO7A1v4

6.2. Grabación de vídeo

A la hora de implementar la grabación de un vídeo se debe tener en cuenta que la API proporciona las siguientes 4 clases:

- **Camera:** Es la clase principal del API para el control de la cámara del dispositivo móvil. Se usa para la toma de fotos y vídeos.

- **SurfaceView**: Esta clase presenta al usuario una previsualización de las imágenes en directo de la cámara.

- **MediaRecorder**: Esta clase se usa para grabar las capturas de vídeo desde la cámara.

- **Intent**: Esta clase permite capturar imágenes o vídeos desde la cámara sin necesidad de tener que usar un objeto Cámara. Un Intent de tipo MediaStore. ACTION_IMAGE_CAPTURE o

ACTION_VÍDEO_CAPTURE define el tipo de captura, si fotografía o vídeo.

Antes de realizar la implementación de la grabación de vídeo, es necesario realizar una configuración en el fichero de manifiesto para tener permisos de uso de la cámara:

```
<uses-permission android:name="android.permission.CAMERA"/>
```

Además, hay que añadir otra configuración para permitir el uso del hardware de la cámara. En ocasiones, si la aplicación va a utilizar una cámara con ciertas características como el flash, se puede especificar añadiendo una nueva declaración en el fichero de manifiesto:

```
<uses-feature android:name="android.hardware.camera"
android:required="false" />
```

En cuanto al almacenamiento, también se ha de añadir permisos para el almacenamiento en la tarjeta SD y permisos para grabar el audio que requiera la grabación de un vídeo con sonido:

```
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission
android:name="android.permission.RECORD_AUDIO"/>
```

La grabación de vídeo se puede realizar de dos formas distintas:

1. **Capturando vídeo con aplicaciones externas:** Son aplicaciones que se encuentran instaladas en el dispositivo y la aplicación Android las usa.

2. **Desarrollando aplicaciones explícitas:** Se desarrolla todo el control de la captura de vídeo, a través de una interfaz de usuario.

Para este segundo caso, los pasos a realizar son los siguientes:

1. **Detectar y acceder a la cámara:** Se debe escribir el código que comprueba la existencia de la cámara y solicite el acceso. La comprobación de si existe la cámara no es necesaria si en el fichero de manifiesto se ha incluido la solicitud de permiso de acceso a la cámara.

2. **Crear una clase de previsualización:** Que herede de SurfaceView e implemente la interfaz SurfaceHolder. Esta clase proporcionará una vista preliminar de las imágenes en directo de la cámara.

3. **Construir la interfaz de previsualización:** Implementar la interfaz de usuario que muestra la previsualización de la cámara.

4. **Crear los listeners para la captura:** Conectar métodos listener a los controles de la interfaz para que el usuario pueda iniciar la captura de la imagen o del vídeo.

5. **Capturar y salvar ficheros:** Escribir el código para guardar la captura en ficheros de salida.

6. **Liberar la cámara:** Una vez que se han realizado las capturas, la aplicación debe liberar el uso de la cámara para que pueda ser usada por otras aplicaciones.

EJEMPLO PRÁCTICO

GRABACIÓN DE VÍDEO

Se va a realizar un ejercicio práctico para la grabación de vídeo. En las siguientes líneas se implementará paso a paso las diferentes llamadas para la grabación de vídeo haciendo uso de la cámara del dispositivo móvil. Es importante seguir cada uno de los pasos en el orden establecido, ya que dependen unos de los otros para realizar la captura.

Detección y acceso a la cámara: la forma de comprobar la existencia sería la siguiente:

```
this.getPackageManager().hasSystemFeature(PackageManager.FEATURE_
CAMERA);
```

No cabe duda de que es mejor realizar la comprobación a través de un método comprobador:

```
private boolean compruebaCameraHardware(Context context) {
 if     (context.getPackageManager().hasSystemFeature
               (PackageManager.FEATURE_CAMERA)) {
 // tiene cámara
 return true;
 } else {
 // no tiene cámara
 return false;
 }
}
```

Un objeto Camera proporciona acceso al hardware de la cámara del dispositivo móvil. Con el método open () se obtiene una referencia del objeto Camera de la cámara principal del móvil. En versiones de API con nivel 9 (Android 2.3) o superior, se dispone del método open(int), permitiendo en el parámetro int el número de la cámara que se quiere acceder en el caso de que el dispositivo móvil contenga más de una.

```
public static Camera obtenCamara() {
 Camera c = null;
 try {
 c = Camera.open(); // obtiene acceso a la cámara
 } catch (Exception e) {
 // no está disponible (está en uso o no existe)
 }
 return c; // devuelve null si cámara no disponible
}
```

Las aplicaciones que manejan directamente la cámara suelen presentar una vista preliminar de la imagen en directo, de tal forma que el usuario puede ver a través de la pantalla la imagen que va a tomar. Esta parte de desarrollo no es necesaria, ya que se podría implementar la aplicación solo con la capacidad de grabación y creación del fichero con la captura del usuario sin la vista de presentación preliminar.

Ejemplo de capturar vídeo mediante la clase MediaRecorder:

```
private void recordVídeo() throws Exception {
 if(recorder!=null){
 recorder.stop();
 recorder.release();
 }
 recorder=new MediaRecorder();
 recorder.setVídeoSource(MediaRecorder.VídeoSource.CAMERA);
 recorder.setAudioSource(MediaRecorder.AudioSource.MIC);

recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
 recorder.setVídeoSize(176,144);
 recorder.setVídeoFrameRate(30);

recorder.setVídeoEncoder(MediaRecorder.VídeoEncoder.MPEG_4_SP);
 recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
 recorder.setOutputFile(OUTPUT_FILE);
 recorder.prepare();
 recorder.start();
}
```

Donde OUTOUT_FILE es la ruta hacia el fichero donde se va a almacenar la captura del vídeo.

De forma resumida, se muestran los pasos a seguir:

1. Capturar el objeto Camera de la aplicación mediante el método camera.open(), el cual devuelve una referencia a ese objeto.

2. Establecer en el objeto Camera realizado el objeto que maneja la previsualización usando el método camera.setPreviewDisplay().

3. Iniciar la previsualización de la imagen en directo de la cámara mediante camera.startPreview().

4. Iniciar el proceso de grabación: Para ello se debe desbloquear el acceso a la cámara a través de camera.unlock(). Seguidamente, establecer el tipo de cámara, audio y vídeo del objeto MediaRecorder a través de los métodos setCamera(), serSudioSource(), setVídeoSource(), setVídeoEncoder() y setAudioEncoder() del objeto en cuestión.

5. Definir el fichero de salida que se generará tras realizar la grabación. Para ello se hace uso del método setOutFile de MediaRecorder.

6. Preparar y comenzar la grabación de vídeo usando los métodos mediaRecorder.prepare() y mediaRecorder.start(), respectivamente.

7. Finalizar la grabación haciendo uso de los métodos stop(), reset() y release(). Liberar la cámara y finalizar la previsualización mediante los métodos camera.lock(), cameraStopPreview() y camera.release().

ENLACE DE INTERÉS

A través del siguiente enlace se pueden obtener ejemplos de una grabación de vídeo.

https://www.codota.com/android/methods/android.media.MediaRecorder/start

VIDEO DE INTERÉS

En los vídeos siguientes tenemos una explicación fantástica de como acceder a la cámara del móvil a través de Android Studio

https://www.youtube.com/watch?v=KX63Wivvpeg&t=684s

https://www.youtube.com/watch?v=Fke4vh06Fes

7. CONTROL Y MONITORIZACIÓN DE LA TRANSMISIÓN POR RTP

RTP (siglas de Realtime Transport Protocol, o Protocolo de Transporte de Tiempo Real), es un protocolo de nivel de sesión para la transmisión de la información en tiempo real. Su utilidad principal, es transmitir audio y vídeo en el marco de sesiones de videoconferencia.

Este protocolo se utiliza, fundamentalmente, en los siguientes escenarios:

- **Streaming:** Distribución de contenidos multimedia por la red, permitiendo que el usuario pueda consumir el produzco en el mismo tiempo que lo descarga.

- **En combinación con el protocolo RTSP** (Real Time Streaming Protocol) el cual controla varios flujos de datos sincronizados, siendo un protocolo no orientado a conexión.

- **Videoconferencias:** Comunicación simultánea y bidireccional de audio y de vídeo, permitiendo mantener reuniones con grupos de usuarios en distintos puntos geográficos (Skype, por ejemplo).

- **Aplicaciones push-to-talk:** O también conocidas como PTT o PPH, para la comunicación half-dúplex, es decir, bidireccional pero no simultánea pulsando un botón para transmitir y soltándolo para recibir.

- **VoIP:** Se utiliza como base para las transmisiones de voz sobre Internet en la que se digitaliza la voz y se transmite en paquetes de datos por la red.

7.1. Streaming

Las aplicaciones Streaming funcionan siguiendo estos criterios:

- El cliente se conecta a un servidor, el cual comienza a enviar el fichero.
- El cliente recibe el fichero y construye un buffer donde va almacenando la información que recibe del servidor.
- Cuando el buffer presenta un mínimo de información almacenada, el cliente comienza a leerla y a mostrarla, continuando con la descarga en curso.
- El sistema está sincronizado de tal manera que a medida que se va descargando, se va visualizando los contenidos.
- Si la conexión sufre retardos en la velocidad, la va utilizando la información del buffer para evitar esos retardos.
- Si la comunicación se corta, se continua con la visualización del contenido almacenado en el buffer. Si se vacía el buffer, se acabará interrumpiendo la visualización del contenido del fichero de descarga.

El streaming, se apoya en los siguientes componentes:

- **Códecs:** Archivos residentes en el PC del cliente que permiten a los reproductores multimedia descifrar o interpretar el contenido de los archivos multimedia.

- **Protocolos:** Como los UDP, RTP y RTSP permiten que la entrega de los paquetes de datos sea más rápida que otros protocolos utilizados en Internet (TCP y HTTP).

- **Buffers:** Para que los datos multimedia que están siendo recibidos se puedan ir almacenando y suplir los posibles retardos que existan en la red y la reproducción no se detenga.

7.2. API de Android

En API de Android en Streaming está implementado en el paquete llamado android.net.rtp. El uso más simple de este paquete consiste en establecer un punto local y otro punto remoto de conexión, estableciendo el streaming entre ambos puntos.

En una aplicación streaming es necesario instancias las siguientes 2 clases:

- **AudioStream**: Especifica un punto final remoto de la conexión y consiste en un enlace de red y en un objeto AudioCodec configurado para la conexión

- **AudioGroup:** Representa el punto local de la conexión para uno o más puntos finales remotos (AudioStream). El AudioGroup mezcla todos los AudioStreams, interactuando con ellos en la comunicación y con los altavoces y micrófono del móvil a la vez.

Para ello es necesario configurar el archivo de manifiesto y otorgar los pertinentes permisos:

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission
android:name="android.permission.RECORD_AUDIO"/>
```

Un objeto AudioGroup es un tipo de hub que mezcla los altavoces, el micrófono y los objetos AudioStreams del streaming pudiendo ser activados o desactivados a través del método setMode(int). El objeto AudioGroup realiza estos pasos:

1. Para cada AudioStream que no esté en modo de sólo envío de paquetes (MODE_SEND_ONLY) descodifica los paquetes que recibe y los almacena en su correspondiente buffer.

2. En segundo lugar, si el micrófono está activo, procesa la grabación de audio y lo almacena en su buffer.

3. Seguidamente, si los altavoces están activos, mezcla todos los buffers de los AudioStream y los reproduce.

4. Por último, para cada AudioStream que no esté en modo de solo recepción (MODE_RECEIVE_ONLY), mezcla todos los buffers y envía los paquetes codificados.

La clase AudioStream, es una subclase de RtpStream y se encarga de gestionar una conexión para transmitir y recibir paquetes mediante el protocolo RTP. Esta transmisión puede ser para que el stream solo transmita paquetes, solo reciba paquetes o transmita y reciba paquetes de forma simultánea.

COMPRUEBA LO QUE SABES

¿Qué protocolos usará la aplicación Skype (usado en videoconferencias) para la transmisión de la comunicación en tiempo real?

8. RESUMEN FINAL

Durante este tema se ha puesto de manifiesto la importancia de conocer el contenido multimedia y sus diferentes medios como textos, imágenes estáticas, dinámicas, audio, vídeo, etc.

Además, se han analizado aquellos formatos más utilizados hoy en día y los que el sistema operativo Android soporta. De igual manera, se han descrito la API y los paquetes que Android Studio ofrece para la implementación de contenido multimedia y acciones sobre los medios como reproducir a través de la Web o de ficheros con el contenido y grabar audio y vídeo.

A modo de resumen, es importante destacar las clases más importantes que se han analizado en este tema:

- La clase MediaPlayer soporta la reproducción de varios formatos y códecs de audio y de vídeo. Permite la reproducción en marcha, pausarla, pararla, retrocederla y avanzarla.

- La clase MediaController proporciona la funcionalidad para realizar las acciones anteriormente comentadas. Se usa junto con la clase VídeoView para la reproducción de vídeos.

- La clase MediaRecorder permite la grabación tanto de audio como de vídeo haciendo uso tanto del micrófono como de la cámara del dispositivo móvil, generando un fichero con la grabación (audio o vídeo) correspondiente.

- El protocolo RTP y las clases AudioGroup y AudioStream para la implementación de la funcionalidad del streaming.